✱ Chapter - 04 ✱

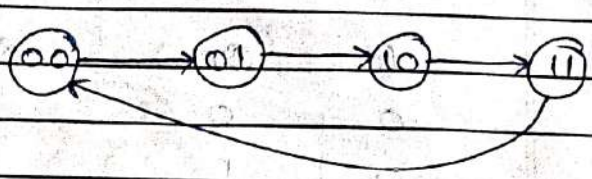
✱ Miscellanous Topics ✱

✱ State Transition Diagram :-

- Graphical [Diagram] representation of all the possible transition from all the possible states of a sequential ckt is called as state transition diagram

eg Q_1Q_0

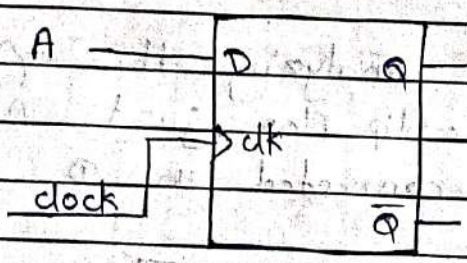
0 - 1 - 2 - 3



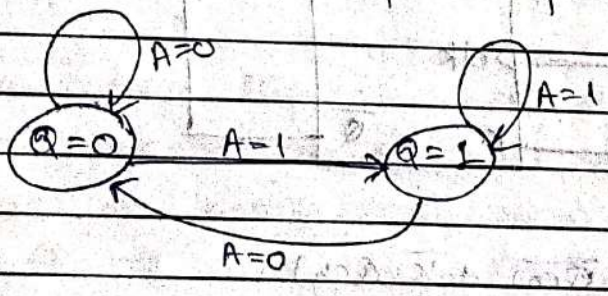
$Q_0(n+1) = \overline{Q_0(n)}$

$Q_1(n+1) = Q_0(n) \oplus Q_1(n)$

eg. ①



→ Here are 1 flip flop so possible states are 2 (0,1)



eqn of D flip flop = $Q(n+1) = D = A$

Here in input only one variable (external) is present. So transition from one state to another is not possible.

When, $Q(n) = 0 \longrightarrow A=0 \rightarrow Q(n+1) = 0$

$\hookrightarrow A=1 \rightarrow Q(n+1) = 1$

$Q(n) = 1 \longrightarrow A=0 \rightarrow Q(n+1) = 0$

$\hookrightarrow A=1 \rightarrow Q(n+1) = 1$

Scanned with OKEN Scanner

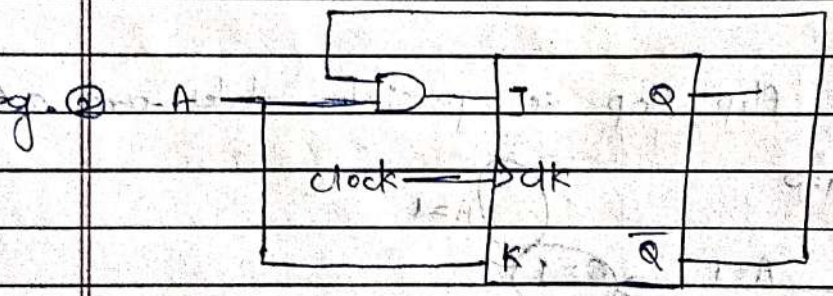
Now if in question, given is state diagram or draw the flip flop diagram
So based on the state diagram calculate the truth table

$Q(n+1)\ [A, Q(n)] = \Sigma(2,3) = A\overline{Q(n)} + AQ(n)$

$\boxed{Q(n+1) = A}$

A	Q(n)	Q(n+1)	
0	0	0	→ starting state Q(n)=0 and we get A=0 then next state state.
0	1	0	
1	0	1	
1	1	1	

$\boxed{Q(n+1) = A}$ if analyzing this eqn is same as D-flip flop just A external variable is connected to D



→

$Q(n+1) = J\overline{Q(n)} + \overline{K}Q(n)$

$= (A.\overline{Q(n)}).\overline{Q(n)} + \overline{A}\,Q(n)$

$= A.\overline{Q(n)} + \overline{A}\,Q(n)$

$\boxed{Q(n+1) = A \oplus Q(n)}$

So there is one FF so possible states are 2 (0 & 1)

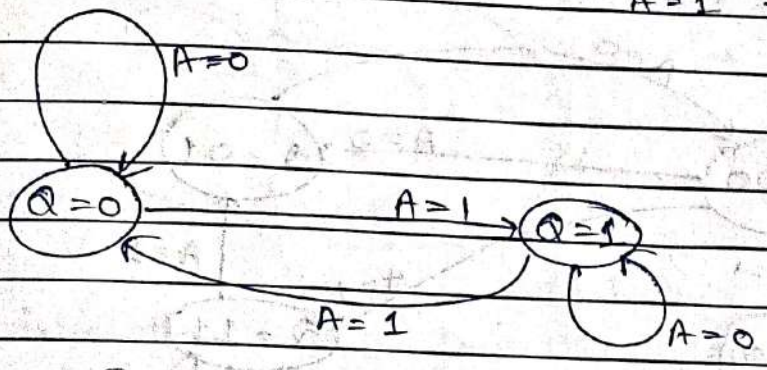
External variable is only 1 i.e. A so possible ~~values~~ transitions from one state to another is 2

$Q(n) = 0 \longrightarrow Q(n+1) = A \rightarrow A=0 \rightarrow Q(n+1) = 0$

$\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\quad \rightarrow A=1 \rightarrow Q(n+1) = 1$

$Q(n) = 1 \longrightarrow Q(n+1) = \bar{A} \rightarrow A=0 \rightarrow Q(n+1) = 1$

$\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\quad \rightarrow A=1 \rightarrow Q(n+1) = 0$

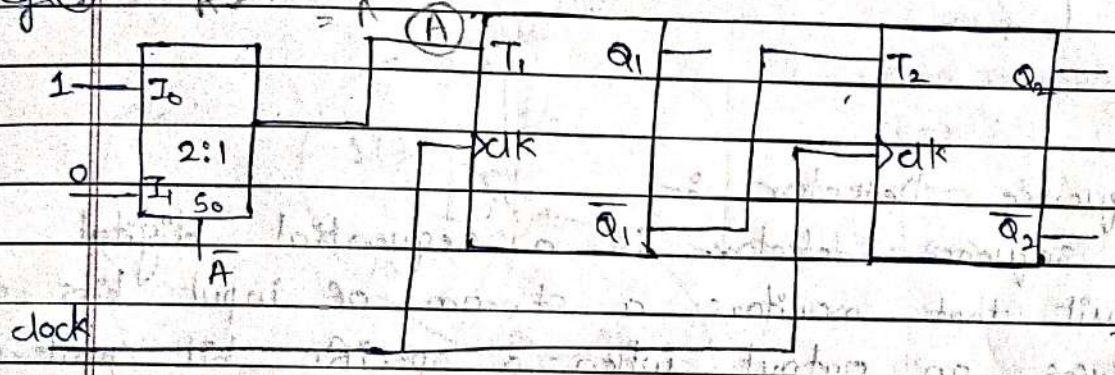


eg. ③

$\bar{S_0} = 0 + S_0 I_1$

$A \cdot 1 + \bar{A} \cdot 0$

$= A$



$\longrightarrow$

Here are two flip flop so possible state are 4 and one external variable so transition possible from one state to another is ~~eight~~ 8

T ff. $\rightarrow Q(n+1) = T \oplus Q(n)$

$Q_1(n+1) = T_1 \oplus Q_1(n) = A \oplus Q_1(n)$

$Q_2(n+1) = T_2 \oplus Q_2(n) = \overline{Q_1(n)} \oplus Q_0(n) = Q_1(n) \odot Q_2(n)$

$$\boxed{\begin{aligned} Q_1(n+1) &= A \oplus Q_1(n) \\ Q_2(n+1) &= Q_1(n) \odot Q_2(n) \end{aligned}}$$

When $Q_1(n) \cdot Q_2(n) = 00 \xrightarrow{Q_1(n)\,Q_2(n)} A=0 \longrightarrow 01$

$\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\quad \rightarrow A=1 \longrightarrow 11$

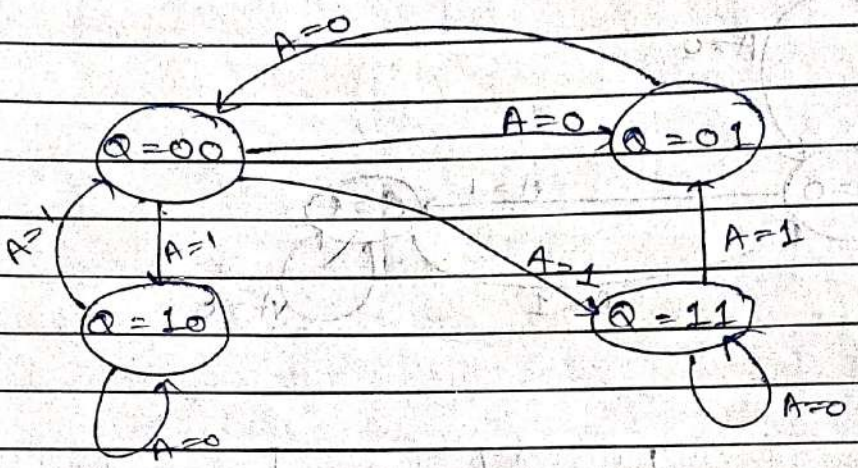
When $Q_1(n) \cdot Q_2(n) = 01 \longrightarrow A=0 \longrightarrow 00$

$\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\quad \rightarrow A=1 \longrightarrow 10$

Scanned with OKEN Scanner

When, $Q_1(n)\,Q_0(n) = 10$ → A=0 → 10 ($Q_1(n)\,Q_0(n)$)
→ A=1 → 00

When, $Q_1(n)\,Q_0(n) = 11$ → A=0 → 11
→ A=1 → 01



* Sequence Detector :-

- A sequence detector is a sequential digital circuit that monitors a stream of input bits and produces an output when a specific bit pattern (sequence) is detected.
- Unlike combinational circuits, a sequence detector remembers previous inputs, which is why it uses flip-flops.

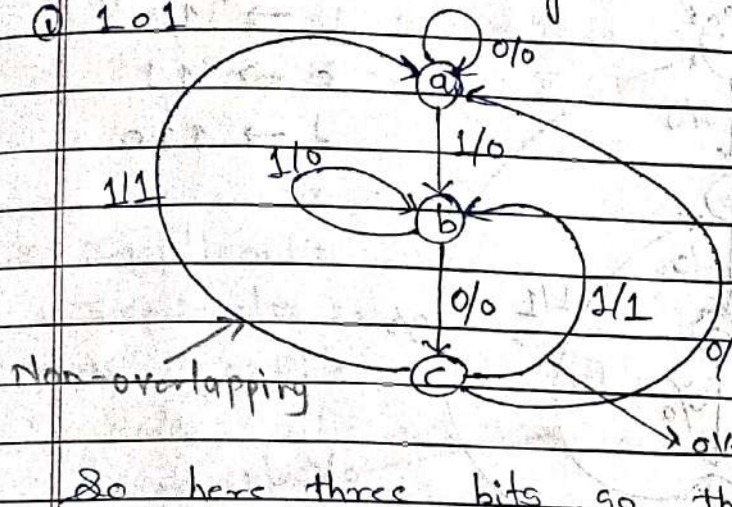
eg we want to detect sequence 101

1 1 0 1 0 1 0 1 0 1 0 1 0 0 0 0 1 0 1 1 0 1 ← overlapping allowed

1 1 0 1 0 1 0 1 0 1 0 1 1 0 0 0 0 1 0 1 ← Non-overlapping sequence detector (or) overlapping not allowed

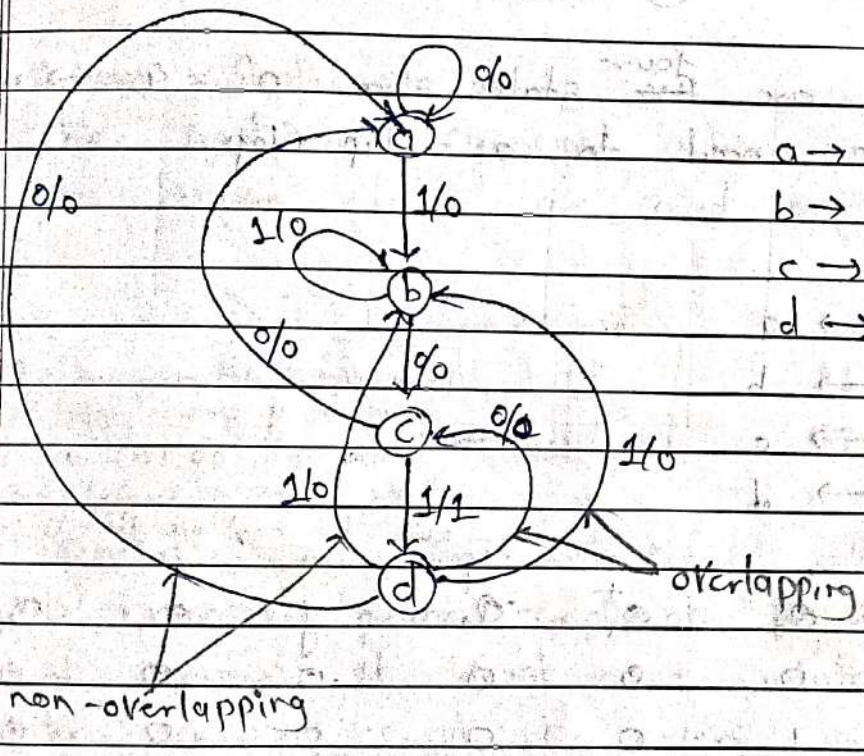
* State Transition diagram for Sequence Detector

① 1 0 1



a → starting state
b → '1'
c → 1 0

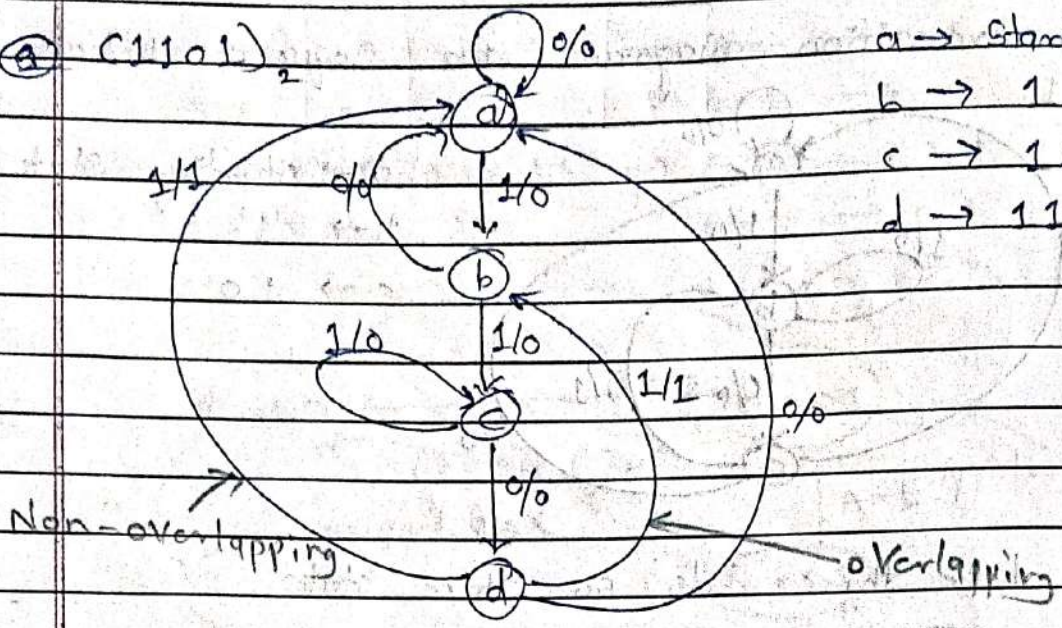
So here three bits so three states



a → starting state
b → '1'
c → '10'
d → '101'

- So diagram shows that input has three bits [101] but states are four (a, b, c, d)

eg (a) $(1101)_2$



a → starting state
b → 1
c → 11
d → 110

So here are ~~five~~ four states so $2^n = 4 \rightarrow n = 2$
So, we need to 2 flip flop

$Q_1\ Q_0$
0 0 → a
0 1 → b
1 0 → c
1 1 → d

Here $(Q_1 Q_0)$ 00 → current state
when at a external input x is 0
next state 00 (a) and at the
time o/p is 0 (y) for D FF
after that next o/p that is same as
i/p for D FF

x	Q_1	Q_0	Q_1^+	Q_0^+	y	D_1	D_0
0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0
0	1	0	1	1	0	1	1
0	1	1	0	0	0	0	0
1	0	0	0	1	0	0	1
1	0	1	1	0	0	1	0
1	1	0	1	0	0	1	0
1	1	1	0	0	1	0	0

(0/1 → Nonoverlap; 0/1 → overlap; 0/1 → Nonoverlap)

$D_1[x, Q_1, Q_0] = \Sigma(0, 5, 6)$

$D_0[x, Q_1, Q_0] = \Sigma(2, 4), 7$ → if overlapping allowed

$y[x, Q_1, Q_0] = \Sigma(7)$

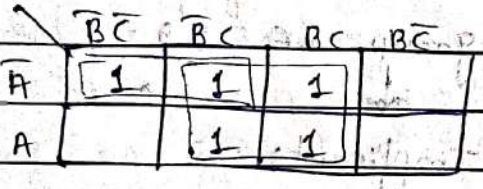
* Implicants :-

- **Implicants :-**

- All possible 1's, all possible pairs, all possible quads, all possible octets, etc. are called as Implicants.

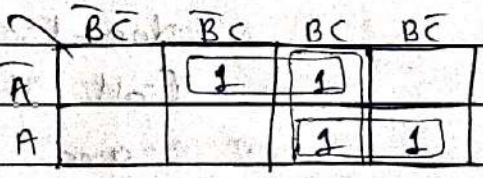
- **Prime Implicants :-**

- The implicants (groups) which can't be converted into bigger group are called as prime implicants.



- **Essential Prime Implicants :-**

- An Essential prime Implicant is a prime Implicant that covers at least one minterm that no other prime Implicant covers.



- EPI are defenetely come in the sol^n of k-map

	$\bar{B}\bar{C}$	$\bar{B}C$	BC	$B\bar{C}$
$\bar{A}$	1	1	1	
A			1	1

PI → $\bar{A}\bar{B}$, $\bar{A}C$, BC, AB

EPI → $\bar{A}\bar{B}$, AB

$f(A,B,C) = \bar{A}\bar{B} + AB + \bar{A}C$ → selective PI

(or) → selective PI

$= \bar{A}\bar{B} + AB + B\bar{C}$

- Non - Essential Prime Implicant :-

- A Non-Essential PI is a prime Implicant whose every minterm is also covered by another prime Implicant.

- Visual Summary

All group of 1's

↓

Implicants

↓

largest possible groups

↓

Prime Implicants

↓

Essential PI [has unique Minterm]

Non-Essential PI [all its minterms are covered by other PIs]

* Number System :-

- Base of a number system : x
  digit → [0 - (x-1)]

- Decimal No. System
  base = 10
  digits = (0 - 9)
  $(5\ 8\ 9)_{10}$ (10^2 10^1 10^0)
  10 → two digit no. in decimal no. system

- $(257)_x$ → valid if $x \geq 8$
  $(a27)_8$ → valid if $0 \leq a \leq 7$

- Imp. number system and their conversion
  ① Decimal no. system
  ↳ base = 10, digit (0-9)
  ② Binary no. system
  ↳ base = 2, digit (0-1)
  ③ Octal no. system
  ↳ base = 8, digit (0-7)
  ④ Hexadecimal no. system
  ↳ base = 16, digit (0-15)
  10 → A
  11 → B
  12 → C
  13 → D
  14 → E
  15 → F

Scanned with OKEN Scanner

Conversion :-

① Anything into decimal

① $(110)_2 = (6)_{10}$ (powers $2^2\,2^1\,2^0$)

② $(136)_8 = [8^2 \times 1 + 8^1 \times 3 + \ldots$ (powers $8^2\,8^1\,8^0$)
$= (94)_{10}$

③ $(2A)_{16} = (16^1 \times 2 + 16^0 \times 10)$ (powers $16^1\,16^0$)
$= (42)_{10}$

④ $(62)_6 = (6^1 \times 6 + 6^0 \times 2)$ (powers $6^1\,6^0$)
$= 36 + 2$
$= (38)_{10}$

② Decimal into Anything

① $(93)_{10} = (1\,0\,1\,1\,1\,0\,1)_2$ (powers $2^6\,2^5\,2^4\,2^3\,2^2\,2^1\,2^0$) $\rightarrow 2^6 = 64 < 93$
$2^7 = 128 > 93$

② $(94)_{10} = (1\,3\,6)_8$ (powers $8^2\,8^1\,8^0$) $\rightarrow 8^2 = 64 < 94$
$8^3 = 512 > 94$

③ $(173)_{10} = (A\;D)_{16}$ (powers $16^1\,16^0$) $\rightarrow 16^1 = 16 < 173$
$16^2 = 256 > 173$

④ $(137)_{10} = (3\;4\;5)_6$ (powers $6^2\,6^1\,6^0$) $\rightarrow 6^2 = 36 < 137$
$6^3 = 216 > 137$

③ Octal no. System

base = 8 $\rightarrow$ (0-7)

000 → 0
001 → 1
010 → 2
011 → 3
100 → 4
101 → 5
110 → 6
111 → 7

Scanned with OKEN Scanner

- Octal to binary

$(27)_8 = (010\ 111)_2$
(2 → 010, 7 → 111)

$(314)_8 = (011\ 001\ 100)_2$
(3 → 011, 1 → 001, 4 → 100)

$(132.52)_8 = (001\ 011\ 010 . 101\ 010)_2$
(1 → 001, 3 → 011, 2 → 010, 5 → 101, 2 → 010)

- Octal to Hexadecimal

first convert to binary and then binary into Hexadec

④ Hexadecimal No. system :-

base = 16 → (0-15)

0000 → 0
0001 → 1
0010 → 2
0011 → 3
0100 → 4
0101 → 5
0110 → 6
0111 → 7
1000 → 8
1001 → 9
1010 → 10 → A
1011 → 11 → B
1100 → 12 → C
1101 → 13 → D
1110 → 14 → E
1111 → 15 → F

- Hexade. to binary

$(2A)_{16} = (0010\ 1010)_2$
(2 → 0010, A → 1010)

$(36D.A94)_{16} =$

$= (0011\ 0110\ 1101 . 1010\ 1001\ 0100)_2$
(3 → 0011, 6 → 0110, D → 1101, A → 1010, 9 → 1001, 4 → 0100)

- Hexadecimal to octal

first convert to binary and then binary to octal

The concept of compliment is develop to the execute subtraction through Addition

DATE

* Concept of Compliments :

base → r

① (r-1)'s compliment → [n-digit max^m number]_r - (num

n-digit number

② r's compliment → (r-1)'s + 1

• 9's and 10's compliment

① $(786)_{10}$

9's → 9 9 9
− 7 8 6
$(2\ 1\ 3)_{10}$

→ Given is 3 digit no so highest 3 digit de no. is 999

10's → 213 + 1 = $(214)_{10}$

② $(12496)_{10}$

9's → 9 9 9 9 9
− 1 2 4 9 6
$(8\ 7\ 5\ 0\ 3)_{10}$

10's → $(87504)_{10}$

③ $(78.64)_{10}$

9's → 9 9 9 9
7 8 6 4
$(2\ 1.\ 3\ 5)_{10}$

10's → 21.36
1
$(21.37)_{10}$ ✓

21.36
1.00
$(22.66)_{10}$ → X

Scanned with OKEN Scanner

• 1's and 2's compliment

① $(1101)_2$ → 1's → 1111 − 1101 = $(0010)_2$

→ 2's → 0010 + 1 = $(0011)_2$

② $(1101101101111)_2$ → 1's → $(0010010010000)_2$

→ 2's → $(0010010010001)_2$

③ (1101) → 2's compliment from right to left upto first 1 copy same as it after 1's compliment remain.

$\underbrace{(0011)}_{\text{comp}}\ \underbrace{}_{\text{same}}$

④ $(001010)_2 \xrightarrow{2's} (110110)_2$

• Note :-

① $(00)_{10}$ → 9's → $(99)_{10}$ → 10's → 99 + 11 = $(00)_{10}$

discard ①00

$(000)_r \xrightarrow{r's} (000)_r$

② $(000)_{10}$ → 9's → $(999)_{10}$ → 10's → $(000)_{10}$

③ $(00)_2$ → 9's → $(11)_2$ → 10's → $(00)_2$

④ $(000)_2$ → 9's → $(111)_2$ → 10's → $(000)_2$

Scanned with OKEN Scanner

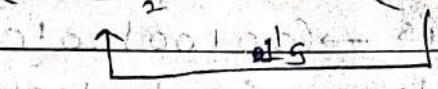
⑤ $(0000)_2 \xrightarrow{2's} (0000)_2$

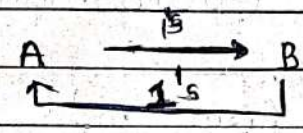
$(1000)_2 \xrightarrow{2's} (1000)_2$

$(10000)_2 \xrightarrow{2's} (10000)_2$

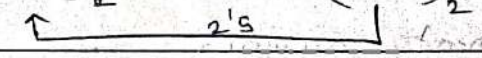
$\underbrace{100\ldots0}_{n\text{-times}} \xrightarrow{2's} (\underbrace{100\ldots0}_{n\text{ times}})_2$

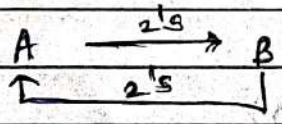
Imp ⑥ $(1101)_2 \xrightarrow{1's} (0010)_2$





⑦ $(1010)_2 \xrightarrow{2's} (0110)_2$





eg. → 8's compliment of $(20)_8$ will be

$(20)_8$

↓ 7's

$\begin{array}{r} 7\ 7 \\ -\ 2\ 0 \\ \hline (5\ 7)_8 \\ +\quad 1 \\ \hline \end{array}$

8's → $(60)_8$

signed → positive (or) Negative

DATE

* Signed No. Representation :-

① Sign magnitude method
② 1's compliment method
③ 2's compliment method

- positive no. representation is same in all of the above method

① Sign Magnitude Method :-

4-bit

0000 → +0	1000 → -0
0001 → +1	1001 → -1
0010 → +2	1010 → -2
0011 → +3	1011 → -3
0100 → +4	1100 → -4
0101 → +5	1101 → +5
0110 → +6	1110 → -6
0111 → +7	1111 → -7

- In this method first digit indicates sign and remaining are no.

$1010 \rightarrow (-2)$, $(1010)_2 \rightarrow -(10)_{10}$

(1 ← sign -ve, 010 ↳ 2)

- The total no. of distinct numbers that can be represented using n bits in sign magnitude method $\boxed{N = (2^n - 1)}$

- Range of the numbers that can be represented using n-bit

$\boxed{-(2^{(n-1)} - 1) \text{ to } +(2^{n-1} - 1)}$

Scanned with OKEN Scanner

② 1's Compliment Method :-

4-bit

0000 → +0	$\xrightarrow{1's}$	1111 →	-0
0001 → +1	$\xrightarrow{1's}$	1110 →	-1
0010 → +2	$\xrightarrow{1's}$	1101 →	-2
0011 → +3	$\xrightarrow{1's}$	1111 →	-3
0100 → +4	$\xrightarrow{1's}$	1011 →	-4
0101 → +5	$\xrightarrow{1's}$	1010 →	-5
0110 → +6	$\xrightarrow{1's}$	1001 →	-6
0111 → +7	$\xrightarrow{1's}$	1000 →	-7

$A \xrightarrow{1's} -A$

could be +ve or -ve number

eg) $(110110)_2 = (-9)_{10}$

$\downarrow 1's$

$(001001)_2 \rightarrow (+9)_{10}$

- The total no. of distinct number that can be represented using n bits in 1's compliment method

$$N = (2^n - 1)$$

- Range of the numbers that can be represented using n-bit

$$-(2^{(n-1)} - 1) \text{ to } +(2^{(n-1)} - 1)$$

③ 2's complimentation Method :-

4-bit

0000 → +0	$\xrightarrow{2's}$	(0000) →	-0
0001 → +1	$\xrightarrow{2's}$	(1111) →	-1
0010 → +2	$\xrightarrow{2's}$	(1110) →	-2
0011 → +3	$\xrightarrow{2's}$	(1101) →	-3
0100 → +4	$\xrightarrow{2's}$	(1100) →	-4
0101 → +5	$\xrightarrow{2's}$	(1011) →	-5
0110 → +6	$\xrightarrow{2's}$	(1010) →	-6
0111 → +7	$\xrightarrow{2's}$	(1001) →	-7

Scanned with OKEN Scanner

$(1000)_2 = -8 + 0 = (-8)_{10}$

(3) $(1011) = (-5) \longrightarrow = -8 + 0 + 2 + 1$
$\downarrow$ 2's $= (-5)_{10}$
$(0101) = +5$

(4) $(1010)_2 = -8 + 0 + 2 + 0 = (-6)_{10}$

- The total no. of distinct number that can be represented using n bits in 2's compliment method $\boxed{N = 2^n}$

- Range of the numbers that can be represented using n-bit

$$\boxed{-(2^{(n-1)}) \text{ to } +(2^{(n-1)} - 1)}$$

- $\boxed{A \xrightarrow{2's} -A}$

+Ve or -Ve

- Note :- (1) $1010 = (-6)_{10}$
$11010 = (-6)_{10}$
$111010 = (-6)_{10}$

(2) $0010 = +2$
$00010 = +2$

(3) Copying MSB any number of times to the left of MSB will not change the number in 2's compliment representation.

eg Represent $(-12)_{10}$ using 8-bit 2's compliment repres
~~$(-10100)_2$~~ $(01100)_2 \longrightarrow +12$
$\downarrow$ 2's
$(10100)_2 \longrightarrow (-12)_{10}$
8+bit $(11110100)_2 \longrightarrow (-12)_{10}$

Scanned with OKEN Scanner

This below examples shows us both addition as well as subtraction in 2's compliment signed no. representation

* Addition and Substraction in 2's compliment signed no. representation

$A - B = A + (-B)$

① $0100 = +4$
$+\ 1010 = -6$
$1110 \longrightarrow (-2)$

② $1100 = -4$
$+\ 0111 = 7$
$(1)0011 \longrightarrow (+3)$
discard

③ $0110 = +6$
$+\ 1110 = -2$
$(1)(0100)_2 = (4)_{10}$
discard

④ $1100 = -4$
$+\ 1110 = -2$
$(1)1010 = (-6)_{10}$
discard

⑤ $0100 = +4$
$+\ 0100 = +4$
$1000 = (-8)$
actual ans = (8)

⑥ $1100 = -4$
$+\ 1010 = -6$
$(1)(0110) = (+6)$
discard
actual ans = +10

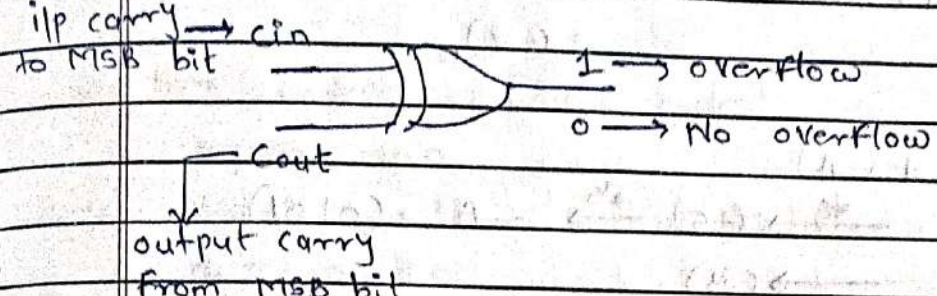
- Example 5th and 6th answer is not correct because their result goes out of range.
  For 4-bit it ranges from (-8) to (+7)
- So this is overflow and this overflow occurs when both no. are positive or negative it is not necessary. In many case if both no. have same sign but its result is correct.

- Condition of overflow :-
  $x \longrightarrow$ sign bit of no. A
  $y \longrightarrow$ sign bit of no. B
  $z \longrightarrow$ sign bit of Result

$$\text{Overflow} = \bar{x} \cdot \bar{y} \cdot z + x \cdot y \cdot \bar{z}$$

Scanned with OKEN Scanner

- When sign of A is negative and sign of B is negative then sign of output should be -ve but if sign of output is positive then overflow occurs.
- When sign of A and B is positive the sign of result should be positive but if sign of result is negative then overflow occurs.



- Overflow Avoid :-
- 2's compliment 4-bit
  -(8) to +(7)
  7 + 7 = 14
  -8 + (-8) = -16
  Addition Result → -16 to 14
  5 bit → -(16) to (15)

- 4 bit sign magnitude or 1's compliment
  (-7) to (+7)
  Addition Result → -14 to 14
  5-bit → (-15) to (+15)

- Note :-
  If we add / subtract two, n-bit numbers in 2's compliment method or 1's compliment method or sign magnitude method, to avoid overflow we require one extra bit in the result i.e. total → (n+1) bits.

* Subtraction in 2's compliment Representation

- $A - B = A + (-B)$

$A = 1011 = -5$

$B = 0011 = 3 \xrightarrow{2's} -B = (1101)_2$

~~0011~~ 1011

~~0011~~ + 1101

discard ① 1000 Result = 1000

$= (-8)_{10}$

- $(B - A) = B + (-A)$

$A = 1011 \xrightarrow{2's} (-5) \xrightarrow{2's} -A = (0101)$

$B = 0011 \longrightarrow (3)$

0 0 1 1

+ 0 1 0 1

(1 0 0 0) $= (+8)_{10}$

* Subtraction of unsigned numbers using 2's compliment :-

$A = (1100)_2 = (12)_{10}$

$B = (0011)_2 = (3)_{10}$

$(A - B) = A + (-B) = A +$ 2's compliment of B

1 1 0 0

1 1 0 1 (2's compliment of B)

① 1 0 0 1 → Result = + (1001) = (+9)

Result is +ve

B − A = B + 2's compliment of A

```
  0011
+ 0100   (2's compliment of A)
  0111
```

no carry generated → Result is −ve

→ − [2's compliment of 0111]
= − [1001]
= − [9]

* BCD codes :- [8421 codes]

0 → 0000
1 → 0001
2 → 0010
3 → 0011
4 → 0100
5 → 0101
6 → 0110
7 → 0111
8 → 1000
9 → 1001

1010 → 10
1011 → 11
1100 → 12
1101 → 13
1110 → 14
1111 → 15
→ invalid BCD code

eg $(25)_{10} = (11001)_2$
$= (0010\ 0101)_{BCD}$

- It is weighted code system

* Excess-3 code :-

- It is unweighted code system

0 → 0011
1 → 0100
2 → 0101
3 → 0110
4 → 0111
5 → 1000
6 → 1001
7 → 1010
8 → 1011
9 → 1100

This is a self complimentry code

0000
0001
0010
1101
1110
1111
→ invalid excess-3 code

eg $(25)_{10} = (0010\ 0101)_{BCD}$
$= (0101\ 1000)_{Excess\ 3}$

Scanned with OKEN Scanner

- Excess -3 code system is the only unweighted code system which is self complimentary in nature
- 2421, 3321, 4311, etc. → are weighted code sys which are self complime in nature

↓

IF it's addition of ~~no~~ weight is equal to 9

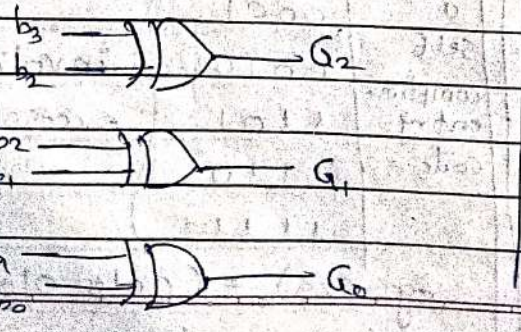
* Gray code :- [Number System]
  - Unweighted Number system
  - UDC → unit distand code
  - When we go subsequently one up or down more then only one bit is changes in gray code.

	Binary	Gray
$(24)_{10}$ =	11000	10100
$(25)_{10}$ =	11001	10101
$(26)_{10}$ =	11010	10111
$(27)_{10}$ =	11011	10110
$(28)_{10}$ =	11100	10010

- Binary to Gray code conversion :-

$b_3 b_2 b_1 b_0 = G_3 G_2 G_1 G_0$

$b_3 = G_3$



eg.

Binary	Gray
1010110	→ 1111101
1101011	→ 1011101

Scanned with OKEN Scanner

- Gray to Binary conversion :-

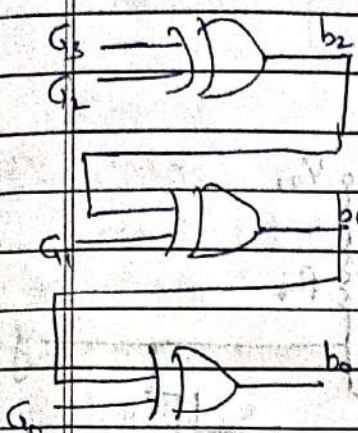
$G_3\ G_2\ G_1\ G_0$ → $b_3\ b_2\ b_1\ b_0$

$b_3 = G_3$

$b_2 = G_3 \oplus G_2$

$b_1 = b_2 \oplus G_1 = G_3 \oplus G_2 \oplus G_1$

$b_0 = b_1 \oplus G_0$
$= G_3 \oplus G_2 \oplus G_1 \oplus G_0$



eg

Gray → Binary

$1011101 \rightarrow 1101110$

$(47)_{10} \rightarrow (110001)_2$

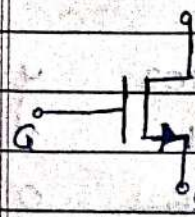
$(101001)_{Gray}$

* MOS logic family :-

The MOS family is family of digital logic circuit built using MOSFETs :

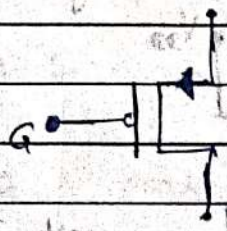
① NMOS
② PMOS
③ CMOS

① NMOS :-



V_G → high → NMOS ON
V_G → low → NMOS off

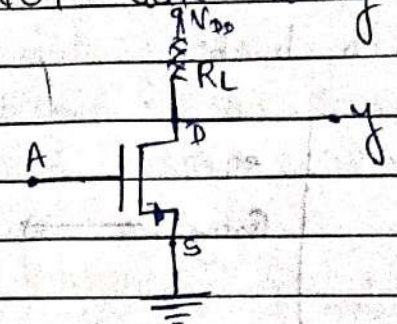
② PMOS



V_G → low → PMOS ON
V_G → high → PMOS off

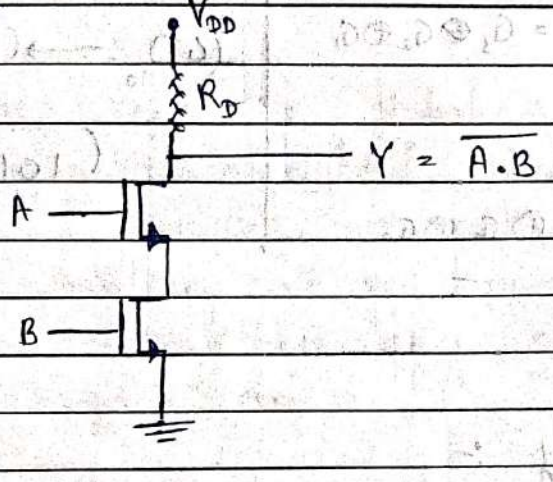
Always we take output from Drain terminal

- NOT Gate using NMOS :-

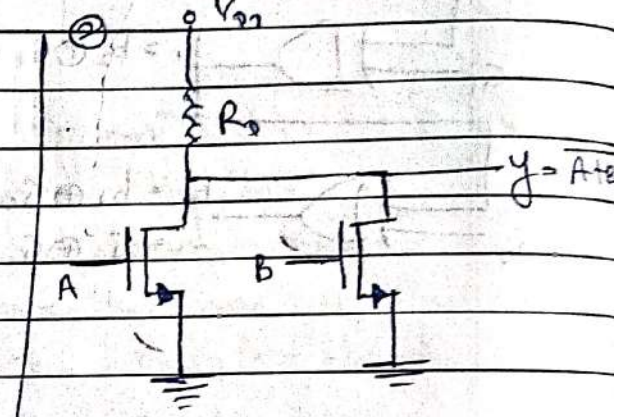


A	NMOS	y
0	OFF	1
1	ON	0

eg. ①



②

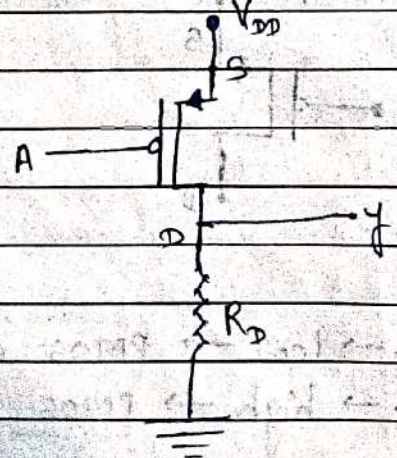


- Notes :-

Series combination of NMOS → equivalent to AND operation

parallel combination of NMOS → equivalent to OR operation

- NOT Gate using PMOS :-



A	PMOS	y
0	ON	1
1	OFF	0

Scanned with OKEN Scanner

- Notes :-

Series combination of PMOS → OR operation

Parallel combination of PMOS → AND operation

NMOS → strong '0'
→ Weak '1'

PMOS → strong '1'
→ Weak '0'

③ CMOS :-

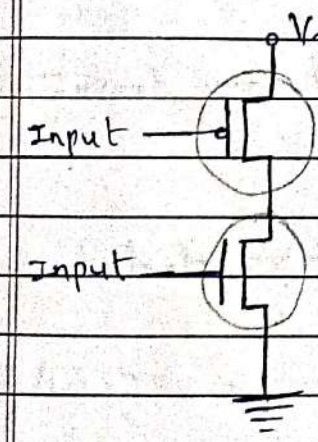
To get strong '0' and strong '1' we use CMOS

- CMOS uses both NMOS and PMOS transistor together

'Complementary' means :→

@PMOS network connects the output to V_{DD}

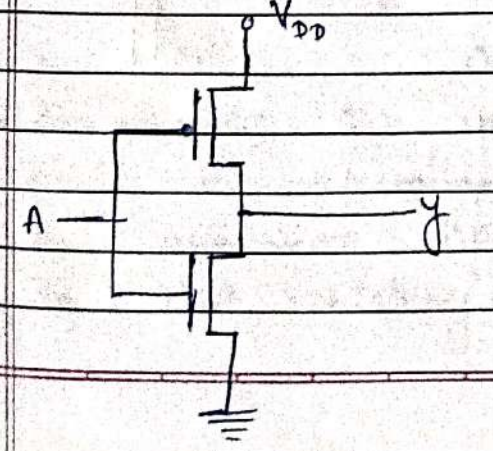
@NMOS network connects the output to GND



Input = 0 :→ PMOS = ON
NMOS = off
o/p → V_{DD}

Input = 1 :→ PMOS = off
NMOS = ON
o/p → GND

- NOT Gate using CMOS :-

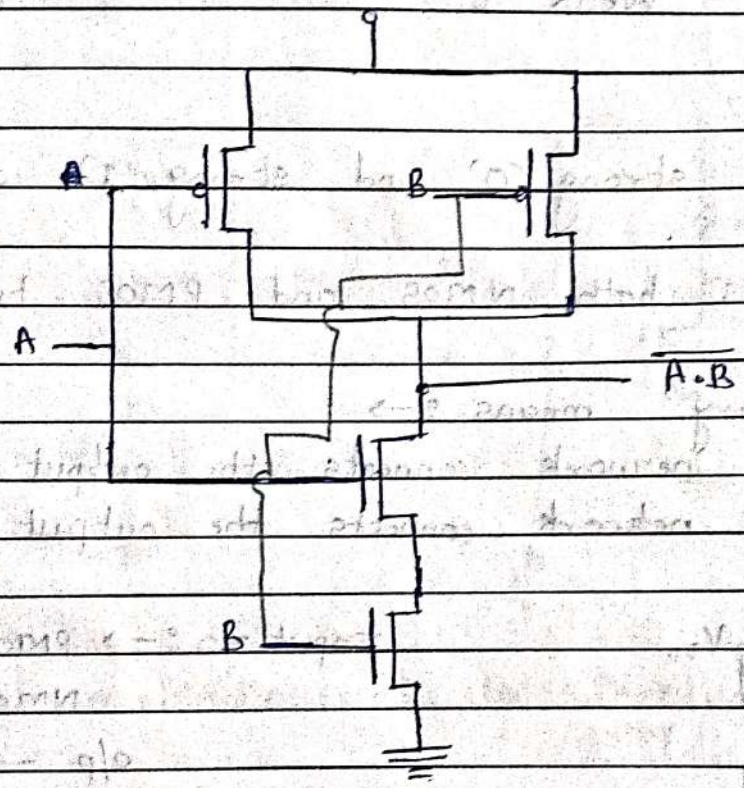


A	PMOS	NMOS	y
0	ON	OFF	1
1	OFF	ON	0

Scanned with OKEN Scanner

- NAND Gate using CMOS :-

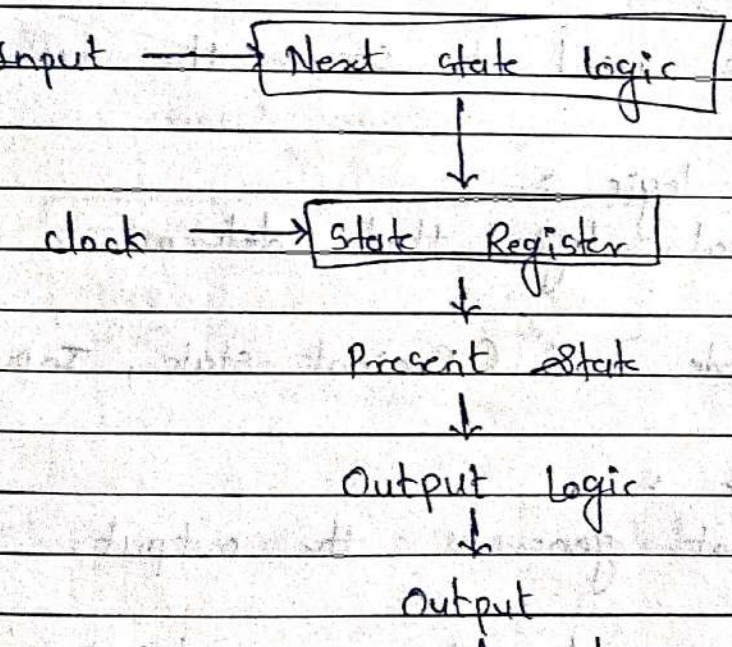
for NAND we need AND means A.B and in NMOS for A.B NMOS are connected in series and in PMOS for A.B PMOS are connected in parallel



* FSM [Finite State Machine] :-
A finite state machine is a sequential digital circuit that has finite no. of states and changes from one state to another depending on the present state and input condition.

- An FSM consist of :-
  1. States
  2. Inputs
  3. Outputs
  4. State transition
  5. clock
  6. State memory [flip-flops]

- Basic representation :



- An FSM is used when a digital system needs to remember what happens previously and perform different operations depending on that history.

① State :
A state represents the current condition o the system

② Present state :
The state in which the FSM is currently operating

③ Next State :
The state that the FSM will enter at the next active clock edge

④ State Transition :
Movement from one state to another
$S_0 \longrightarrow S_1$

⑤ State Register :
Flip-flops used to store the current state

⑥ Next-state logic :
Combinational logic that determines the next state
Next state = f (Present state, Input)

⑦ Output logic :
logic that generates the output

Its behaviour depends on whether the FS is Moore or Mealy

- FSM Types :-
  1. Moore FSM
  2. Mealy FSM

1. Moore FSM

In moore FSM, the o/p depends only on the present state.

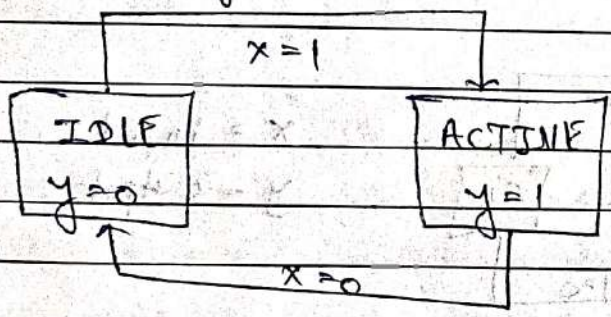
$$y = f(s)$$

∴ y = output
s = present state

Input → Next state logic → State Register → Present state → output logic → output

- The output does not directly depend on input.

- State Diagram



- The output is associated with state

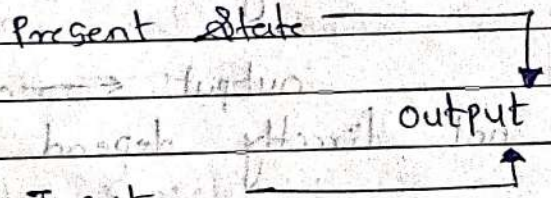
② Mealy FSM :-

In Mealy FSM, the output depends on both :- ① Present state
② Input

$$y = f(S, x)$$

y = output
S = present state
x = input

- Block Diagram :-



- Mealy example.

